

Análisis y Diseño de Algoritmos

Dr. José Martínez Carranza

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Introducción al Curso	2
2. Funciones asintóticas	6
3. Continuación de funciones asintóticas	14

1. Introducción al Curso

18 de agosto de 2026

Resumen rápido. Sesión inicial: temario, criterios de evaluación y datos de contacto. El profesor comparó bubble sort contra quick sort a partir del video de Obama y de ahí saltó a por qué sigue valiendo la pena estudiar estructuras de datos y algoritmos en la era de la IA. Cerró con computable, decidible e indecidible, y con P contra NP.

Info general

Temario:

1. Introducción
2. Notación asintótica
3. Recurrencias
4. Ordenamiento
5. Algoritmos voraces
6. Divide y vencerás
7. Programación dinámica
8. *(falta anotar este tema)*
9. Algoritmos aleatorizados
10. Teoría de complejidad

Criterios de evaluación:

- 90 % exámenes, tres parciales de 30 % cada uno
- 10 % asistencia

Contacto y material:

- Sitio del curso: <http://ccc.inaoep.mx/~carranza/alg.html>
- Correo: carranza@inaoep.mx

Ojito...

Todo correo al profesor lleva ALG2026 en el asunto. Si no, se pierde en el SPAM.

El objetivo del curso es desarrollar pensamiento computacional para resolver problemas. No se trata tanto de programar sino de analizar y diseñar algoritmos eficientes.

Ordenamiento (por arribita)

A raíz del vídeo de Obama, en el cual le preguntan cómo ordenar un millón de enteros, se da una embarradita de algoritmos de ordenamiento. Son los dos ejemplos con los que el profesor mostró que dos algoritmos correctos para el mismo problema no cuestan lo mismo.

Bubble sort

La idea es ir burbujeando los elementos. Se recorre el arreglo comparando pares vecinos y se intercambian cuando están en desorden, así que en cada pasada el mayor de los que quedan sube hasta su posición final. Se caracteriza por usar 2 ciclos anidados, lo que denota su complejidad cuadrática.

Mejor caso, caso promedio y peor caso son los tres $\mathcal{O}(n^2)$. La versión simple recorre los dos ciclos anidados completos sin importar cómo venga el arreglo, así que un arreglo ya ordenado le cuesta lo mismo que uno al revés.

Quick sort

Parte el problema. Elige un pivote, acomoda a la izquierda lo menor y a la derecha lo mayor, y se llama a sí mismo sobre cada mitad. Como las dos mitades ya no dependen una de la otra, el algoritmo es paralelizable.

Algoritmo	Mejor	Promedio	Peor	Memoria
Bubble sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Quick sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$

Tabla 1: Costo de los dos algoritmos vistos en clase.

Idea clave

Dos algoritmos igual de correctos para el mismo problema pueden diferir en órdenes de magnitud. Escoger el algoritmo es una decisión de diseño, no un detalle de implementación.

Por qué estudiar DSA

Saber como funcionan los algoritmos y como usar las estructuras de datos nos permite

- Romper problemas en pasos pequeños.
- Elegir la estructura de datos correcta.
- Analizar la complejidad en tiempo y en memoria, porque siempre se busca la solución óptima.
- Comparar diseños y sus trade-offs.

Se espera que al terminar el curso uno pueda determinar la estrategia correcta para resolver un problema y dar soluciones óptimas, escalables y eficientes en tiempo y espacio.

Conceptos clave

- **Algoritmo:** secuencia finita y no ambigua de pasos que, a partir de una entrada, produce una salida en un número finito de operaciones.
- **Problema computable:** el problema para el que sí se puede estructurar un programa que obtenga un resultado. Saber cuál es el último número natural no lo es: no hay tal número, así que no hay programa que lo devuelva.
- **Problema decidable:** existe un algoritmo que, para cualquier entrada, se detiene y entrega una respuesta.
- **Problema indecidible:** no existe tal algoritmo. Ninguna solución general lo resuelve.

Verificación

Si desarrollo un algoritmo que va a resolver un problema, la pregunta inmediata es cómo sé que esa solución es válida. Esa es la verificación, y es un paso aparte de escribir el código.

DSA en tiempos de la IA

Conocer estructuras de datos y algoritmos sirve para analizar el código que la IA genera. El vibe coding sirve para prototipar rápido, pero *abstraction eventually leaks*: tarde o temprano hay que bajar al detalle que la abstracción escondía. Los dos sirven mejor en conjunto.

Ojito...

El código generado siempre lo juzga un humano, para detectar áreas de oportunidad y cosas por optimizar. Sin ese criterio no hay forma de saber si lo que salió es bueno.

Cómo convertirse en experto en DSA

Antes era a la antigua: tirar código hasta que la intuición apareciera sola y uno supiera qué estructura de datos usar en cada caso. Eso ya cambió. Ahora se aprende la teoría primero, se entienden los fundamentos, se estudian las matemáticas detrás de DSA y luego se practica con ejemplos.

El ciclo de trabajo que espera el profesor:

1. Intentarlo por mi cuenta.
2. Pedir una pista, no la solución.

3. Implementar y probar.
4. Preguntarle a la IA por mi razonamiento.
5. Explicarle la solución a la IA.
6. Resolver un problema similar sin IA.

Idea clave

La práctica hace al maestro. La IA entra como quien revisa, no como quien resuelve.

El santo grial: P contra NP

P es la clase de los problemas que se resuelven en tiempo polinomial. NP es la de los problemas cuya solución propuesta se *verifica* en tiempo polinomial, aunque encontrarla parezca costar tiempo exponencial. Todo problema de P está en NP, porque resolverlo ya es una forma de verificarlo. La pregunta abierta es la contraria: si algo se verifica rápido, ¿también se resuelve rápido?

Demostrar $P = NP$ implicaría que miles de problemas que hoy se atacan por aproximación tienen algoritmo eficiente y nadie lo ha encontrado. Demostrar $P \neq NP$ cerraría la puerta de una vez. Nadie ha hecho ninguna de las dos.

La IA agentica vamos para allá... (La IA que resuelve problemas por sí misma, es decir, puede fijar metas, planificar y ejecutar tareas de forma humana para alcanzar un objetivo específico, con poca supervisión humana) pero eso nos podría hacer tratar todo como una caja negra... pero eso no es lo ideal. Siempre queremos entender cómo funcionan las cosas, y el estudio de DSA nos da esas herramientas.

2. Funciones asintóticas

20 de agosto de 2026

Resumen rápido. En esta clase vimos una intro a la notación asintótica y a las funciones de crecimiento, así como un ejemplo/técnica de cómo se puede analizar la complejidad de un algoritmo. Finalizamos la clase con un poco de historia, hablando sobre el Debate Lighthill de 1973, "The general purpose robot is a mirage"

Qué es una función?

Definición 2.1. Una función es una relación entre dos conjuntos, donde a cada elemento del primer conjunto (*dominio*) le corresponde un único elemento del segundo conjunto (*codominio*). conjunto.

Conceptos importantes:

- Dominio: conjunto de entrada, todos los valores de entrada permitidos, por ejemplo, los reales.
- Codominio: conjunto de llegada, donde viven las salidas.
- Imagen: conjunto de llegada que realmente se obtiene. Subconjunto del codominio.

Ojito...

Rango y recorrido son sinónimos de imagen.

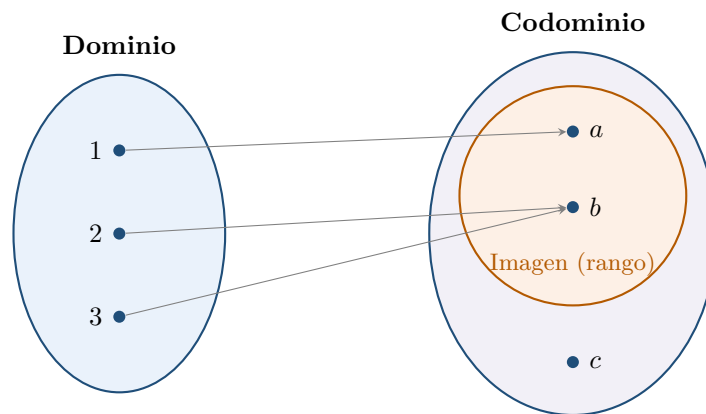


Figura 1: El elemento c pertenece al codominio pero no a la imagen.

Una función de los reales a los reales por ejemplo se denota con

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

que básicamente dice que es una función que toma un real y devuelve un real. Si esta función se define como $f(x) = x^2$, entonces el dominio es $\mathbb{R}$, el codominio es $\mathbb{R}$ y la imagen es $\mathbb{R}_{\geq 0}$.

Analisis Asintotico

Supongamos que tenemos un arreglo con n elementos desordenados y queremos ordenarlos, como ya sabemos tenemos varias opciones que hacen la misma tarea, pero se comportaran diferente dependiendo del tamaño del arreglo.

Ordenamiento Burbuja

El arreglo va creciendo y como se observa en la grafica de abajo, el tiempo que tarda en ordenarse tambien

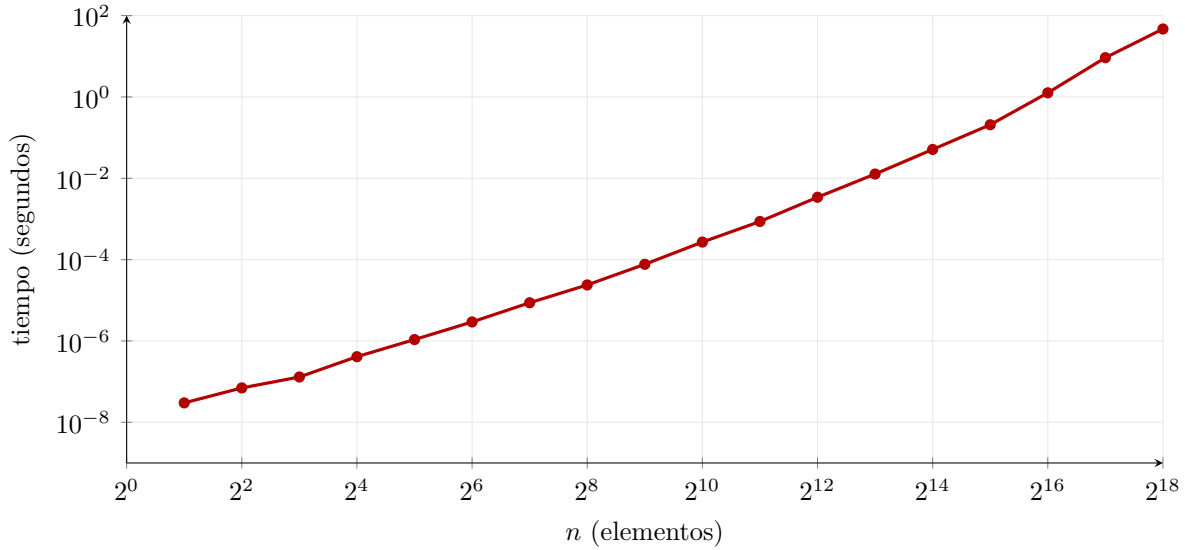


Figura 2: Tiempo de bubble sort sobre arreglos aleatorios de 2^1 a 2^{18} elementos.

Estos datos son discretos, no son continuos pero podemos aproximar una funcion, interpolar, usar alguna tecnica de algoritmos evolutivos o hasta con intuicion

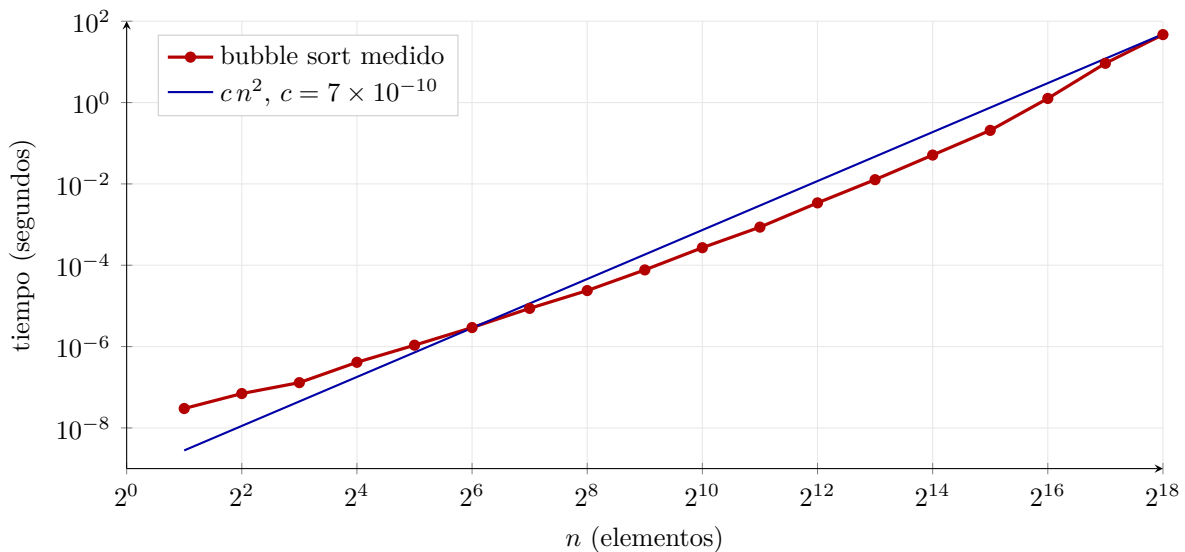


Figura 3: Tiempos medidos de bubble sort contra la curva cn^2 .

Se puede ver que la curva cn^2 está encima de los puntos medidos, así que podemos decir que el tiempo de bubble sort es $\mathcal{O}(n^2)$.

Pendiente por resolver

La curva cn^2 es una cota superior, pero no es la mejor. ¿Cómo encontrar la mejor cota?... LO digo pq como que no acota todos los puntos y ademas se ve como una linea recta pero quizas sea por la escala

UPDATE: No toca los puntos por lo dicho abajo, para valores pequeños es irrelevante, el problema es que no se eligio un buen n_0 pero le pregunto al profe.

Quick sort

Ahora el mismo experimento con quick sort, los datos (resultados) se muestran a continuacion:

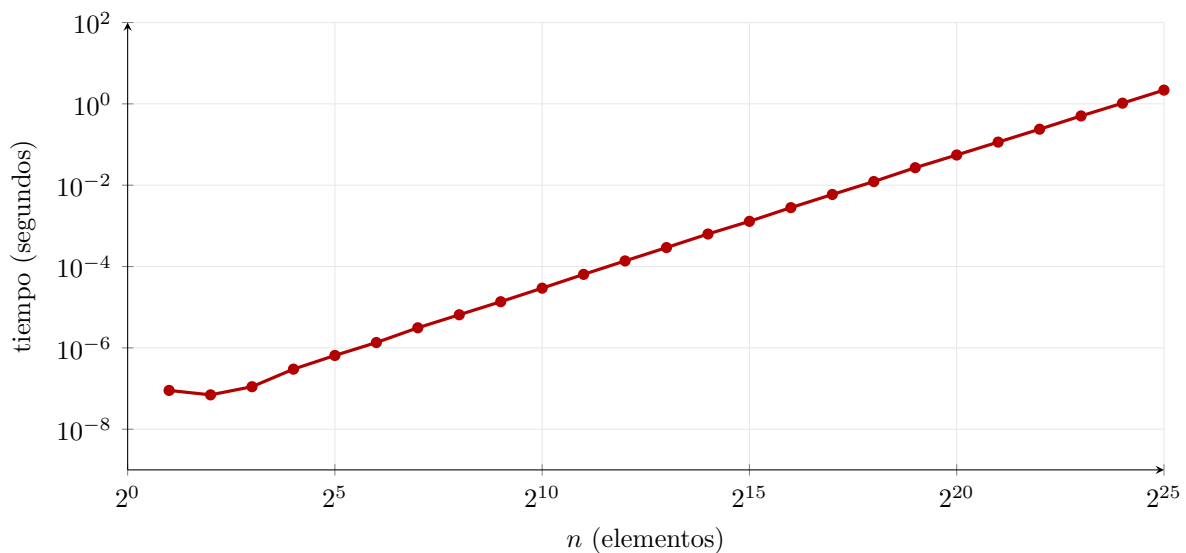


Figura 4: Tiempo de quick sort sobre arreglos aleatorios de 2^1 a 2^{25} elementos.

Al tratar de determinar la función analítica a ojo de buen cubero...

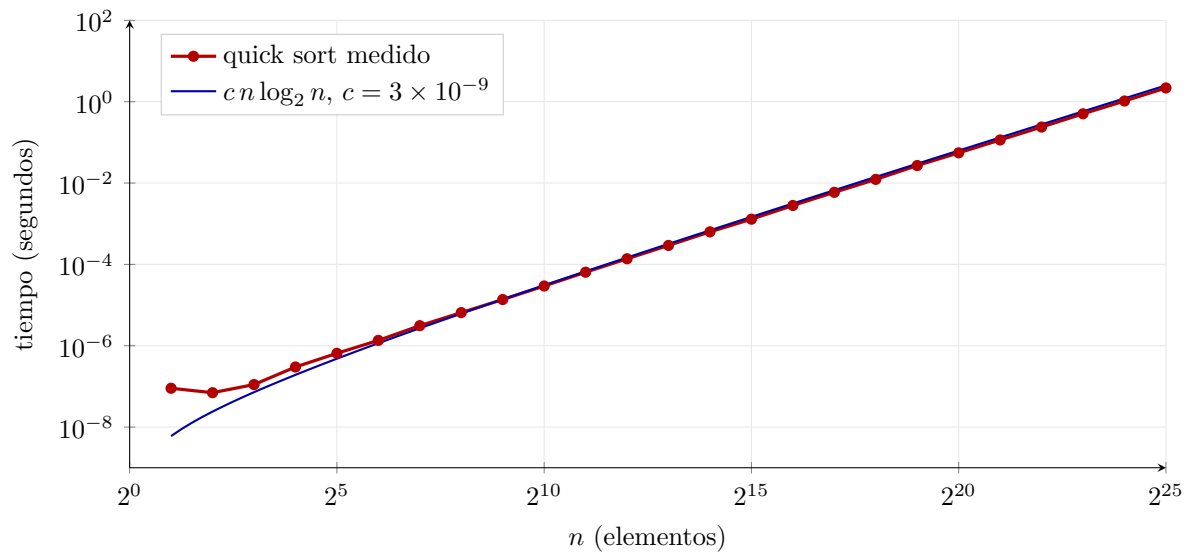


Figura 5: Tiempos medidos de quick sort contra la curva $cn \log_2 n$.

La curva $cn \log_2 n$ acota mas o menos bien los puntos medidos, asi que podemos decir que el tiempo de quick sort es $\mathcal{O}(n \log n)$.

Pendiente por resolver

Igual que en burbuja, por que no acota todos los puntos??

UPDATE: No toca los puntos por lo dicho abajo, para valores pequeños es irrelevante, el problema es que no se eligio un buen n_0 pero le pregunto al profe.

Brubuja vs Quick

Al comparar ambos algoritmos en la sig grafica...

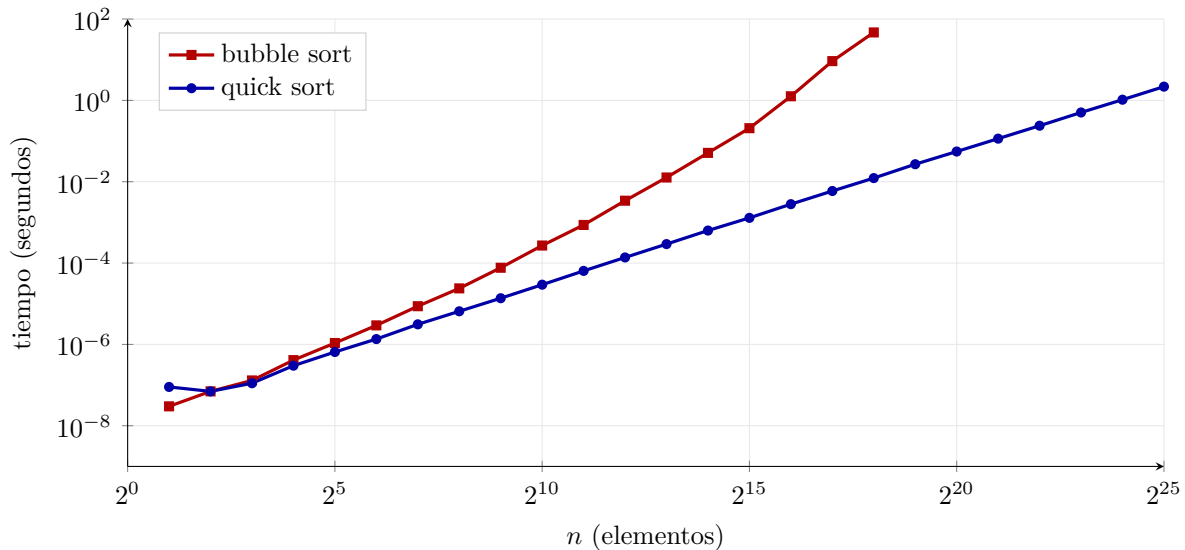


Figura 6: Bubble sort contra quick sort sobre los mismos arreglos. segundos.

Se puede ver que quick sort es mucho mas rapido que burbuja, y la diferencia se hace mas evidente a medida que el tamaño del arreglo crece. Por ejemplo, burbuja lo detuve en 2^{18} elementos porque ya tardaba demasiado, mientras que quick sort lo corrí hasta 2^{25} elementos y seguía siendo razonable. Entonces podemos concluir que quick sort es mucho mas eficiente que burbuja.

Ojito...

Se puede hacer esta comparacion ya que los arreglos que se usan para probar ambos algoritmos fueron los mismos y se testeo en la misma PC. El codigo se encuentra en <https://github.com/zxxz456/inaoe/tree/main/ada/dummy/ComparaOrdenamientos.c>

Notacion Big O

En los ejemplos de arriba mencione que el tiempo de bubble sort es $\mathcal{O}(n^2)$ y el tiempo de quick sort es $\mathcal{O}(n \log n)$.

Lo que quiere decir esto es que la funcion empirica encontrada del ALGORITMO de ordenamiento burbuja $f(n)$, pertenece al conjunto de funciones o a la familia de funciones $\mathcal{O}(n^2)$

Y analogamente la funcion empirica encontrada del ALGORITMO de ordenamiento quick sort $f(n)$, pertenece al conjunto de funciones o a la familia de funciones $\mathcal{O}(n \log n)$

De forma mas general, nuestra funcion empirica $f(n)$ la queremos modelar como una funcion continua $c \cdot g(n)$ y dada esta funcion $g(n)$ denotamos $\mathcal{O}(g(n))$ al conjunto de funciones tales que existen constntes positivas c y n_0 tales que para todo $n \geq n_0$ se cumple que $f(n) \leq c \cdot g(n)$

Definición 2.2. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\mathcal{O}(g(n))$ si existen constantes positivas c y n_0 tales que

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$\mathcal{O}(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0 \text{ tales que } 0 \leq f(n) \leq c \cdot g(n) \quad \forall n \geq n_0 \}.$$

Y lo que dice basicamente es que $f(n)$ esta en el conjunto de funciones $\mathcal{O}(g(n))$ y que crece a lo mucho como esa $g(n)$

Ejemplo 2.3. Sean $f(n) = 2n^2 + 3n + 1$ y $g(n) = n^2$. Tomando $c = 6$ y $n_0 = 1$ se cumple

$$2n^2 + 3n + 1 \leq 6n^2 \iff 4n^2 - 3n - 1 \geq 0 \iff (4n + 1)(n - 1) \geq 0,$$

lo cual es cierto para todo $n \geq 1$. Por lo tanto $f(n) \in \mathcal{O}(n^2)$.

El par (c, n_0) no es único: con $c = 3$ la desigualdad $2n^2 + 3n + 1 \leq 3n^2$ equivale a $n^2 - 3n - 1 \geq 0$, que se satisface a partir de $n \geq \frac{3+\sqrt{13}}{2} \approx 3.30$, de modo que $n_0 = 4$ también funciona. La definición solo exige que *exista* al menos un par de constantes.

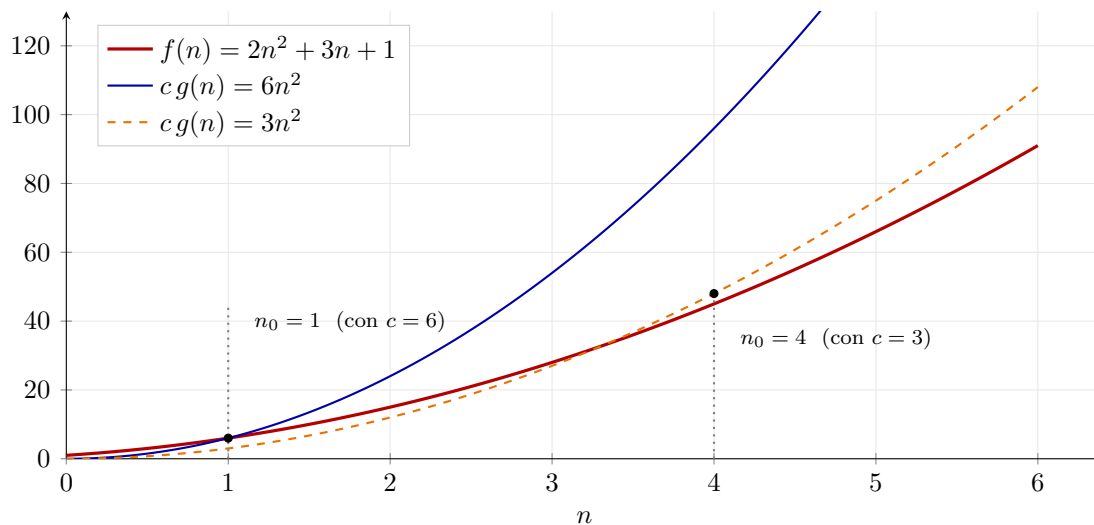


Figura 7: Para $n \geq 1$ la curva $6n^2$ domina a $f(n)$; a partir de $n \geq 4$ basta con $3n^2$.

Ojito...

El "para todo $n \geq n_0$ " es lo que hace que la def sea asintótica. El comportamiento en entradas chiquitas es irrelevante, la curva f puede estar por encima de $cg(n)$ tanto como sea antes del umbral.

Ojito...

La Big O da una cota superior q no tiene que ser ajustada. Es perfectamente cierto que $n = \mathcal{O}(n^5)$, aunque sea una cota muy floja. Cuando quieres afirmar que la cota es ajustada por arriba y por abajo, usas $\Theta(g(n))$, definida con dos constantes: $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$.

Ojito...

Como $\mathcal{O}(g(n))$ es un conjunto, lo correcto sería escribir $f(n) \in \mathcal{O}(g(n))$. La costumbre de escribir $f(n) = \mathcal{O}(g(n))$ es un abuso de notación establecido, y por eso el signo no es simétrico: puedes escribir $2n^2 = \mathcal{O}(n^3)$, pero nunca $\mathcal{O}(n^3) = 2n^2$.

Crecimiento de funciones

Determinar la eficiencia asintótica de un algoritmo pasa por modelarlo: encontrar la función que describe su costo y ver contra qué se puede acotar. Muchas funciones admiten cota asintótica, y en la práctica casi todo termina comparándose contra tres familias: polinomios, logaritmos y exponenciales.

Operación elemental

Definición 2.4. Una operación elemental es aquella cuyo tiempo de ejecución está acotado por una constante que no depende del tamaño de la entrada. Una comparación, una asignación o una suma cuentan como una sola, sin importar qué tan grande sea n .

Un algoritmo casi nunca ejecuta un solo tipo de operación. Sumar dos matrices de $n \times n$ hace sumas, pero también compara índices, incrementa contadores y accede a memoria. Se escoge una operación representativa, la que se ejecuta al menos tan seguido como cualquier otra, y se cuenta esa nada más: las demás quedan absorbidas por un factor constante

Principio de invarianza

Dos implementaciones del mismo algoritmo, en lenguajes distintos o sobre máquinas distintas, no tardan lo mismo, pero sus tiempos difieren a lo mucho por un factor constante. Esa es la razón de que la constante se descarte: lo que cambia al cambiar de máquina es ella, mientras que la clase asintótica es propiedad del algoritmo y sobrevive al cambio

Mejor caso, peor caso y caso promedio

El costo no depende solo de n , también de cómo venga la entrada, así que se analizan tres escenarios. Quick sort es el ejemplo de la tabla 1: en promedio cuesta $\mathcal{O}(n \log n)$, pero si el pivote cae siempre en un extremo las particiones quedan desbalanceadas y degenera a $\mathcal{O}(n^2)$. La burbuja simple no distingue, sus tres casos son $\mathcal{O}(n^2)$ porque recorre los ciclos completos sin importar cómo venga el arreglo

Idea clave

Siempre conviene optimizar. Es la única forma de llegar a los límites de lo que las arquitecturas actuales pueden dar

3. Continuación de funciones asintóticas

25 de agosto de 2026

Resumen rápido. Continuación de la clase anterior. Las propiedades de las tres notaciones, con la regla del límite que las decide de un jalón. $\Omega(\cdot)$ como cota inferior, $\Theta(\cdot)$ como las dos a la vez y el teorema que las relaciona, más los dos ejemplos completos. Cerramos con las notaciones chicas, los órdenes de complejidad y los consejos para contar operaciones elementales.

Propiedades de $\mathcal{O}(\cdot)$

Sean f, g, h, f_1 y f_2 funciones positivas.

1. Para cualquier función f se tiene $f \in \mathcal{O}(f)$.
2. $f \in \mathcal{O}(g) \Rightarrow \mathcal{O}(f) \subset \mathcal{O}(g)$.
3. $\mathcal{O}(f) = \mathcal{O}(g) \iff f \in \mathcal{O}(g)$ y $g \in \mathcal{O}(f)$.
4. Si $f \in \mathcal{O}(g)$ y $g \in \mathcal{O}(h) \Rightarrow f \in \mathcal{O}(h)$.
5. Si $f \in \mathcal{O}(g)$ y $f \in \mathcal{O}(h) \Rightarrow f \in \mathcal{O}(\min(g, h))$.
6. **Regla de la suma:** si $f_1 \in \mathcal{O}(g)$ y $f_2 \in \mathcal{O}(h) \Rightarrow f_1 + f_2 \in \mathcal{O}(\max(g, h))$.
7. **Regla del producto:** si $f_1 \in \mathcal{O}(g)$ y $f_2 \in \mathcal{O}(h) \Rightarrow f_1 \cdot f_2 \in \mathcal{O}(g \cdot h)$.
8. Si existe $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = k$, entonces según el valor de k :
 - Si $k \neq 0$ y $k < \infty$, entonces $\mathcal{O}(f) = \mathcal{O}(g)$.
 - Si $k = 0$, entonces $f \in \mathcal{O}(g)$, es decir $\mathcal{O}(f) \subset \mathcal{O}(g)$, pero se verifica que $g \notin \mathcal{O}(f)$.

Idea clave

La propiedad del límite es la herramienta práctica. En lugar de buscar c y n_0 a mano, se calcula el cociente f/g y se ve a dónde tiende: si tiende a una constante distinta de cero, las dos funciones son del mismo orden; si tiende a cero, f crece estrictamente más despacio.

Notación Omega

$f(n) = \Omega(g(n))$ quiere decir que $g(n)$ es una cota asintótica inferior de $f(n)$.

Definición 3.1. Dada una función $g(n)$, denotamos al conjunto de funciones $\Omega(g(n))$ de la siguiente forma:

$$\Omega(g(n)) = \{ f : \mathbb{N} \rightarrow \mathbb{R}^+ \mid \exists c \text{ constante positiva y } n_0 : 0 < c g(n) \leq f(n), \forall n \geq n_0 \}.$$

Ojito...

$f(n) \in \Omega(g(n))$ si y solo si $g(n) \in \mathcal{O}(f(n))$. Las dos notaciones son la misma relación vista desde cada lado.

Propiedades de $\Omega(\cdot)$

1. Para cualquier función f se tiene $f \in \Omega(f)$.
2. $f \in \Omega(g) \Rightarrow \Omega(f) \subset \Omega(g)$.
3. $\Omega(f) = \Omega(g) \iff f \in \Omega(g) \text{ y } g \in \Omega(f)$.
4. Si $f \in \Omega(g)$ y $g \in \Omega(h) \Rightarrow f \in \Omega(h)$.
5. Si $f \in \Omega(g)$ y $f \in \Omega(h) \Rightarrow f \in \Omega(\max(g, h))$.
6. **Regla de la suma:** si $f_1 \in \Omega(g)$ y $f_2 \in \Omega(h) \Rightarrow f_1 + f_2 \in \Omega(g + h)$.
7. **Regla del producto:** si $f_1 \in \Omega(g)$ y $f_2 \in \Omega(h) \Rightarrow f_1 \cdot f_2 \in \Omega(g \cdot h)$.
8. Si existe $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = k$, entonces según el valor de k :
 - Si $k \neq 0$ y $k < \infty$, entonces $\Omega(f) = \Omega(g)$.
 - Si $k = 0$, entonces $g \in \Omega(f)$, es decir $\Omega(g) \subset \Omega(f)$, pero se verifica que $f \notin \Omega(g)$.

Ojito...

En $\mathcal{O}(\cdot)$ la propiedad 5 usa el mínimo y la regla de la suma el máximo. En $\Omega(\cdot)$ la 5 usa el máximo y la suma da $g + h$. No son simétricas, conviene no copiarlas de memoria.

Por qué importan las cotas inferiores

Una cota inferior dice cuándo ya no se puede mejorar dentro de un modelo de cómputo. El ejemplo canónico es el ordenamiento por comparaciones: en el modelo general de comparaciones, cualquier algoritmo de ordenamiento necesita del orden de $n \log n$ comparaciones en el peor caso, o sea $\Omega(n \log n)$.

Idea clave

Si un algoritmo ya logra $\mathcal{O}(n \log n)$ y el problema tiene cota inferior $\Omega(n \log n)$, entonces ese algoritmo es asintóticamente óptimo en ese modelo. $\Omega(\cdot)$ contesta cuál es el trabajo mínimo que hay que hacer.

Notación Theta

$f(n) = \Theta(g(n))$ quiere decir que $c_2 g(n)$ y $c_1 g(n)$ son las cotas asintóticas de $f(n)$, superior e inferior respectivamente. Diremos que $f(n) \in \Theta(g(n))$ si $f(n)$ pertenece tanto a $\mathcal{O}(g(n))$ como a $\Omega(g(n))$.

Definición 3.2.

$$\Theta(g(n)) = \{ f : \mathbb{N} \rightarrow \mathbb{R}^+ \mid \exists c_1, c_2 \text{ constantes positivas, } n_0 : 0 < c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0 \}.$$

Propiedades de $\Theta(\cdot)$

1. Para cualquier función f se tiene $f \in \Theta(f)$.
2. $f \in \Theta(g) \Rightarrow \Theta(f) \subset \Theta(g)$.
3. $\Theta(f) = \Theta(g) \iff f \in \Theta(g) \text{ y } g \in \Theta(f)$.

4. Si $f \in \Theta(g)$ y $g \in \Theta(h) \Rightarrow f \in \Theta(h)$.
5. **Regla de la suma:** si $f_1 \in \Theta(g)$ y $f_2 \in \Theta(h) \Rightarrow f_1 + f_2 \in \Theta(\max(g, h))$.
6. **Regla del producto:** si $f_1 \in \Theta(g)$ y $f_2 \in \Theta(h) \Rightarrow f_1 \cdot f_2 \in \Theta(g \cdot h)$.
7. Si existe $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = k$, entonces según el valor de k :
 - Si $k \neq 0$ y $k < \infty$, entonces $\Theta(f) = \Theta(g)$.
 - Si $k = 0$, entonces los órdenes exactos de f y g son distintos.

Teorema 3.3. $f(n) = \Theta(g(n))$ si y solo si $f(n) = \mathcal{O}(g(n))$ y $f(n) = \Omega(g(n))$.

Demostración. De ida es inmediato: la definición de $\Theta(\cdot)$ contiene las dos desigualdades, así que c_2 y n_0 sirven de testigo para $\mathcal{O}(\cdot)$, y c_1 y n_0 para $\Omega(\cdot)$.

De regreso, si $f \in \mathcal{O}(g)$ existen c_2 y n_1 con $f(n) \leq c_2 g(n)$ para $n \geq n_1$, y si $f \in \Omega(g)$ existen c_1 y n_2 con $c_1 g(n) \leq f(n)$ para $n \geq n_2$. Tomando $n_0 = \max\{n_1, n_2\}$ las dos valen a la vez a partir de n_0 , que es justo la definición de $\Theta(g(n))$. $\square$

Ejemplos completos

Ejemplo 3.4 (Theta). Considere $f(n) = \frac{1}{2}n^2 - 3n$. Como es un polinomio cuadrático, su estructura general es $an^2 + bn + c$, y para n muy grande el término an^2 domina al resto. Se propone entonces $g(n) = n^2$ y se demuestra que $f(n) \in \Theta(n^2)$.

Hay que encontrar c_1 , c_2 y n_0 tales que

$$0 < c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2, \quad \text{y dividiendo entre } n^2, \quad 0 < c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2.$$

Se analiza por casos. Para $c_1 \leq \frac{1}{2} - \frac{3}{n}$, como c_1 es constante positiva se necesita $0 < \frac{1}{2} - \frac{3}{n}$, lo que da $n > 6$. Con $n_0 = 7$ queda $c_1 \leq \frac{1}{2} - \frac{3}{7} = \frac{1}{14}$, así que se toma $c_1 = \frac{1}{14}$.

Para $\frac{1}{2} - \frac{3}{n} \leq c_2$, cuando $n \rightarrow \infty$ el lado izquierdo tiende a $\frac{1}{2}$, así que $c_2 = \frac{1}{2}$.

Con $c_1 = \frac{1}{14}$, $c_2 = \frac{1}{2}$ y $n_0 = 7$ se cumple que $f(n) \in \Theta(n^2)$.

Ejemplo 3.5 (Big O). Considere $f(n) = 2n^2 + 3n + 1$, otra vez un polinomio cuadrático donde an^2 domina, así que se propone $g(n) = n^2$.

Hay que encontrar c y n_0 tales que

$$0 < 2n^2 + 3n + 1 \leq c n^2, \quad \text{o sea} \quad 0 < 2 + \frac{3}{n} + \frac{1}{n^2} \leq c.$$

Si $n \rightarrow \infty$ el lado izquierdo tiende a 2, y si $n = 1$ vale 6. Por tanto, con $c = 6$ y $n_0 = 1$ se demuestra que $f(n) \in \mathcal{O}(n^2)$.

Una cota puede ser correcta y a la vez imprecisa

Sea $T(n) = 3n^2 + 7n + 20$. Todas estas afirmaciones son matemáticamente ciertas:

$$T(n) \in \mathcal{O}(n^2), \quad T(n) \in \mathcal{O}(n^3), \quad T(n) \in \mathcal{O}(n^4), \quad T(n) \in \mathcal{O}(2^n).$$

Cada una de esas funciones se puede escalar por alguna constante positiva de modo que termine quedando por encima de $T(n)$. Por eso Big O por sí sola no identifica la verdadera tasa de crecimiento. $\Theta(n^2)$ sí: dice que el crecimiento cuadrático es el orden asintótico correcto, salvo factores constantes.

Ojito...

$\mathcal{O}(\cdot)$ no es sinónimo de peor caso, ni $\Omega(\cdot)$ de mejor caso. Son relaciones matemáticas de acotamiento. Primero se elige qué función de tiempo se está analizando y después se le aplica la notación que sea. Se puede decir $T_{\text{mejor}}(n) = \Theta(n)$, o $T_{\text{peor}}(n) = \Omega(n \log n)$, sin ninguna contradicción.

Notaciones chicas

$f(n) = o(g(n))$ quiere decir que $g(n)$ es una cota superior de $f(n)$ que no es asintóticamente justa (*tight*). Y $f(n) = \omega(g(n))$, que $g(n)$ es una cota inferior de $f(n)$ que tampoco lo es.

La distinción precisa no es tanto lo justo o no justo, sino el cuantificador sobre la constante:

- $f(n) = \mathcal{O}(g(n)) \iff \exists c > 0, \exists n_0 : f(n) \leq c g(n), \forall n \geq n_0.$
- $f(n) = o(g(n)) \iff \forall c > 0, \exists n_0 : f(n) < c g(n), \forall n \geq n_0.$

Con $\Omega(\cdot)$ y $\omega(\cdot)$ pasa lo mismo, volteando la desigualdad. Las grandes permiten que las dos funciones sean del mismo orden; las chicas lo prohíben.

Propiedad	$\mathcal{O}(g(n))$	$o(g(n))$
Tipo de cota superior	No estricta	Estricta
Condición sobre c	$\exists c > 0$	$\forall c > 0$
Permite el mismo orden	Sí	No
Puede ser $\Theta(g(n))$	Sí	No
Cociente $f(n)/g(n)$	Acotado basta	Tiende a 0

Tabla 2: Big O contra o chica.

El punto de vista del límite

Cuando el límite existe, el cociente deja la distinción clarísima:

$$f = o(g) \iff \frac{f(n)}{g(n)} \rightarrow 0, \quad f = \Theta(g) \iff \frac{f(n)}{g(n)} \rightarrow L \text{ con } 0 < L < \infty, \quad f = \omega(g) \iff \frac{f(n)}{g(n)} \rightarrow \infty.$$

$\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ son relaciones más amplias: solo piden una cota por factor constante, no un límite específico. Por ejemplo $n^2 \neq o(n^2)$, porque el cociente vale 1 y no 0; en cambio $n = o(n^2)$, porque $n/n^2 = 1/n \rightarrow 0$.

Jerarquía de crecimiento

$$1 \ll \log n \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n$$

Leída con o chica: $n = o(n \log n)$, $n \log n = o(n^2)$, $n^2 = o(n^3)$. Y al revés con omega chica: $n \log n = \omega(n)$, $n^2 = \omega(n \log n)$, $n^3 = \omega(n^2)$.

Notación	Crecimiento	Tipo de cota
$o(g(n))$	Estrictamente más lento	Superior estricta
$\mathcal{O}(g(n))$	No más rápido	Superior
$\Theta(g(n))$	Del mismo orden	Ajustada
$\Omega(g(n))$	No más lento	Inferior
$\omega(g(n))$	Estrictamente más rápido	Inferior estricta

Tabla 3: Las cinco relaciones asintóticas de un vistazo.

Idea clave

$o(\cdot) < \mathcal{O}(\cdot) - \Theta(\cdot) - \Omega(\cdot) < \omega(\cdot)$. Las grandes admiten crecimiento del mismo orden, las chicas lo prohíben. Esa es toda la diferencia.

Orden de complejidad

La familia $\mathcal{O}(f(n))$, $\Omega(f(n))$, $\Theta(f(n))$ define un orden de complejidad, y como representante del orden se elige la función $f(n)$ más sencilla de la familia. Se identifican diferentes familias:

Orden	Nombre
$\mathcal{O}(c)$	Constante
$\mathcal{O}(\log n)$	Logarítmico
$\mathcal{O}(n)$	Lineal
$\mathcal{O}(n \log n)$	Casi lineal
$\mathcal{O}(n^2)$	Cuadrático
$\mathcal{O}(n^3)$	Cúbico
$\mathcal{O}(n^c)$	Polinómico de grado c
$\mathcal{O}(2^n)$	Exponencial
$\mathcal{O}(n!)$	Factorial

Tabla 4: Órdenes de complejidad.

Consejos para identificar $f(n)$

Son los consejos para dar con la $f(n)$ que representa el número de operaciones elementales (OE) de un algoritmo.

Consejo 1: la base

Se asume que el tiempo de una OE es de orden 1, o sea que la constante c del principio de invarianza se toma como 1 por fines prácticos. El tiempo de ejecución de una secuencia de instrucciones, elementales o no, se obtiene sumando los tiempos de cada instrucción.

Instrucción case

Para `switch(n)` con casos $S_1, \dots, S_k$:

$$f(n) = f(c) + \max\{f(S_1), \dots, f(S_k)\},$$

donde $f(c)$ considera el tiempo de comparar n contra cada uno de los casos.

Instrucción if

Para if C then S_1 else S_2 :

$$f(n) = f(C) + \max\{f(S_1), f(S_2)\}.$$

```

1  if (n == 0)
2      for (i = 0; i < n; i++)
3          r += i;
4  else
5      r = 2;
```

Instrucción while

Para while (c) { S }:

$$f(n) = f(c) + (\#iteraciones) \cdot (f(c) + f(S)).$$

Ojito...

Las instrucciones **for**, **repeat** y **loop** son equivalentes a un **while**, así que se analizan con la misma fórmula.

Llamado a funciones no recursivas

El tiempo de una llamada $F(A_1, \dots, A_s)$ es 1 por el llamado, más el tiempo de evaluar los parámetros, más el tiempo de ejecutar el cuerpo de la función:

$$f(A_1, \dots, A_s) = 1 + f(A_1) + \dots + f(A_s) + f(S).$$

Funciones recursivas

El tiempo de ejecución de una función recursiva se calcula a través de la solución de funciones de recurrencia, que es el tema que sigue.